Tabla Usuarios

```sql
CREATE TABLE usuarios (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    avatar_url VARCHAR(500) DEFAULT 'default_avatar.png',
    biografia TEXT,
    ubicacion VARCHAR(100),
    fecha_registro DATETIME DEFAULT CURRENT_TIMESTAMP,
    ultimo_login DATETIME,
    activo BOOLEAN DEFAULT TRUE,
    rol ENUM('usuario', 'moderador', 'admin') DEFAULT 'usuario',
    INDEX idx_email (email),
    INDEX idx_fecha_registro (fecha_registro)
);
```

Tabla Categorías

```sql
CREATE TABLE categorias (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    descripcion TEXT,
    icono VARCHAR(50),
    color VARCHAR(7) DEFAULT '#f4630c',
    orden INT DEFAULT 0,
    activa BOOLEAN DEFAULT TRUE,
    UNIQUE KEY uk_nombre (nombre)
);

-- Datos iniciales de categorías
INSERT INTO categorias (nombre, descripcion, icono) VALUES
('CPU', 'Procesadores', '⚡'),
('GPU', 'Tarjetas Gráficas', '🎮'),
('RAM', 'Memoria RAM', '💾'),
('Refrigeración', 'Refrigeración Líquida', '❄️'),
```

```
('Alimentación', 'Fuentes de Alimentación', '🔌'),
('Motherboard', 'Placas Base', '🧩'),
('Storage', 'Almacenamiento', '💾'),
('Case', 'Cajas/Gabinetes', '🏠');
```

Tabla Publicaciones

```sql
CREATE TABLE publicaciones (
    id INT PRIMARY KEY AUTO_INCREMENT,
    usuario_id INT NOT NULL,
    categoria_id INT NOT NULL,
    titulo VARCHAR(200) NOT NULL,
    componente VARCHAR(100) NOT NULL,
    contenido TEXT NOT NULL,
    valoracion INT CHECK (valoracion >= 1 AND valoracion <= 5),
    likes INT DEFAULT 0,
    fecha_publicacion DATETIME DEFAULT CURRENT_TIMESTAMP,
    fecha_actualizacion DATETIME ON UPDATE CURRENT_TIMESTAMP,
    estado ENUM('publicado', 'borrador', 'eliminado') DEFAULT 'publicado',
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE CASCADE,
    FOREIGN KEY (categoria_id) REFERENCES categorias(id),
    INDEX idx_usuario (usuario_id),
    INDEX idx_categoria (categoria_id),
    INDEX idx_valoracion (valoracion),
    INDEX idx_fecha_publicacion (fecha_publicacion),
    FULLTEXT INDEX idx_busqueda (titulo, componente, contenido)
);
```

Tabla Comentarios

```sql
CREATE TABLE comentarios (
    id INT PRIMARY KEY AUTO_INCREMENT,
    publicacion_id INT NOT NULL,
    usuario_id INT NOT NULL,
    contenido TEXT NOT NULL,
    fecha_comentario DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (publicacion_id) REFERENCES publicaciones(id) ON DELETE CASCADE,
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE CASCADE,
    INDEX idx_publicacion (publicacion_id),
```

```sql
    INDEX idx_usuario (usuario_id),
    INDEX idx_fecha (fecha_comentario)
);
```

Tabla Likes

```sql
CREATE TABLE likes (
    id INT PRIMARY KEY AUTO_INCREMENT,
    publicacion_id INT NOT NULL,
    usuario_id INT NOT NULL,
    tipo ENUM('like', 'dislike') DEFAULT 'like',
    fecha DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (publicacion_id) REFERENCES publicaciones(id) ON DELETE CASCADE,
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE CASCADE,
    UNIQUE KEY uk_like_usuario (publicacion_id, usuario_id),
    INDEX idx_publicacion_usuario (publicacion_id, usuario_id)
);
```

Tabla Seguidores

```sql
CREATE TABLE seguidores (
    id INT PRIMARY KEY AUTO_INCREMENT,
    usuario_id INT NOT NULL,
    seguidor_id INT NOT NULL,
    fecha_seguimiento DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE CASCADE,
    FOREIGN KEY (seguidor_id) REFERENCES usuarios(id) ON DELETE CASCADE,
    UNIQUE KEY uk_seguimiento (usuario_id, seguidor_id),
    INDEX idx_usuario_seguido (usuario_id),
    INDEX idx_seguidor (seguidor_id)
);
```

Tabla Chat

```sql
CREATE TABLE chat_mensajes (
    id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    emisor_id INT NOT NULL,
    receptor_id INT NOT NULL,
    mensaje TEXT NOT NULL,
    fecha_envio DATETIME DEFAULT CURRENT_TIMESTAMP,
    leido BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (emisor_id) REFERENCES usuarios(id) ON DELETE CASCADE,
    FOREIGN KEY (receptor_id) REFERENCES usuarios(id) ON DELETE CASCADE,
    INDEX idx_emisor (emisor_id),
    INDEX idx_receptor (receptor_id),
    INDEX idx_conversacion (LEAST(emisor_id, receptor_id), GREATEST(emisor_id,
receptor_id), fecha_envio)
);
```

Tabla Notificaciones

```sql
CREATE TABLE notificaciones (
    id INT PRIMARY KEY AUTO_INCREMENT,
    usuario_id INT NOT NULL,
    tipo ENUM('like', 'comentario', 'seguidor', 'sistema') NOT NULL,
    contenido TEXT NOT NULL,
    enlace VARCHAR(500),
    fecha_creacion DATETIME DEFAULT CURRENT_TIMESTAMP,
    leida BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE CASCADE,
    INDEX idx_usuario (usuario_id),
    INDEX idx_leida (leida),
    INDEX idx_fecha (fecha_creacion)
);
```

Vista para Estadísticas de Usuario

```sql
CREATE VIEW vista_estadisticas_usuario AS
SELECT
    u.id,
    u.nombre,
    COUNT(DISTINCT p.id) AS total_publicaciones,
    SUM(p.likes) AS total_likes_recibidos,
    COUNT(DISTINCT s.seguidor_id) AS total_seguidores,
    COUNT(DISTINCT s2.usuario_id) AS total_siguiendo,
```

```sql
    COUNT(DISTINCT c.id) AS total_comentarios,
    COALESCE(AVG(p.valoracion), 0) AS valoracion_promedio
FROM usuarios u
LEFT JOIN publicaciones p ON u.id = p.usuario_id
LEFT JOIN seguidores s ON u.id = s.usuario_id
LEFT JOIN seguidores s2 ON u.id = s2.seguidor_id
LEFT JOIN comentarios c ON u.id = c.usuario_id
WHERE u.activo = TRUE
GROUP BY u.id, u.nombre;
```

Procedimiento Almacenado para Crear Publicación

```sql
DELIMITER $$

CREATE PROCEDURE crear_publicacion(
    IN p_usuario_id INT,
    IN p_categoria_id INT,
    IN p_titulo VARCHAR(200),
    IN p_componente VARCHAR(100),
    IN p_contenido TEXT,
    IN p_valoracion INT
)
BEGIN
    DECLARE v_publicacion_id INT;

    -- Insertar la publicación
    INSERT INTO publicaciones (
        usuario_id,
        categoria_id,
        titulo,
        componente,
        contenido,
        valoracion
    ) VALUES (
        p_usuario_id,
        p_categoria_id,
        p_titulo,
        p_componente,
        p_contenido,
        p_valoracion
    );
```

```sql
    -- Obtener el ID de la publicación creada
    SET v_publicacion_id = LAST_INSERT_ID();

    -- Actualizar estadísticas del usuario
    UPDATE usuarios
    SET ultimo_login = CURRENT_TIMESTAMP
    WHERE id = p_usuario_id;

    -- Seleccionar la publicación creada
    SELECT
        p.*,
        u.nombre as usuario_nombre,
        u.avatar_url,
        c.nombre as categoria_nombre
    FROM publicaciones p
    JOIN usuarios u ON p.usuario_id = u.id
    JOIN categorias c ON p.categoria_id = c.id
    WHERE p.id = v_publicacion_id;
END$$

DELIMITER ;
```

Trigger para Actualizar Likes

```sql
DELIMITER $$

CREATE TRIGGER actualizar_likes_publicacion
AFTER INSERT ON likes
FOR EACH ROW
BEGIN
    IF NEW.tipo = 'like' THEN
        UPDATE publicaciones
        SET likes = likes + 1
        WHERE id = NEW.publicacion_id;
    END IF;

    -- Crear notificación para el dueño de la publicación
    INSERT INTO notificaciones (usuario_id, tipo, contenido)
    SELECT
```

```
        p.usuario_id,
        'like',
        CONCAT(u.nombre, ' le dio like a tu publicación: "', LEFT(p.titulo, 50), '"')
    FROM publicaciones p
    JOIN usuarios u ON u.id = NEW.usuario_id
    WHERE p.id = NEW.publicacion_id;
END$$

DELIMITER ;
```

4. Configuración del Archivo de Conexión (config.py)

python

```python
# config.py
import os
from dotenv import load_dotenv

load_dotenv()

class Config:
    # Configuración de base de datos
    DB_HOST = os.getenv('DB_HOST', 'localhost')
    DB_USER = os.getenv('DB_USER', 'root')
    DB_PASSWORD = os.getenv('DB_PASSWORD', '')
    DB_NAME = os.getenv('DB_NAME', 'verdict_db')
    DB_PORT = int(os.getenv('DB_PORT', 3306))

    # Configuración de la aplicación
    SECRET_KEY = os.getenv('SECRET_KEY', 'clave-secreta-para-desarrollo')
    UPLOAD_FOLDER = 'static/uploads'
    MAX_CONTENT_LENGTH = 16 * 1024 * 1024  # 16MB máximo

    # Configuración de sesión
    SESSION_TYPE = 'filesystem'
    SESSION_PERMANENT = False

    @property
    def DATABASE_URI(self):
        return
f"mysql+pymysql://{self.DB_USER}:{self.DB_PASSWORD}@{self.DB_HOST}:{self.DB_PORT}/{sel
f.DB_NAME}"
```

```
config = Config()
```